Data structures

AMORTIZED ANALYSIS & BINARY HEAPS

Amortized analysis

Amortized analysis averages the cost of operations over a sequence, smoothing out rare expensive operations.

Array-Doubling Example:

- **Best case**: O(1) *(regular insertion, no resizing)*
- **Worst case**: O(n) *(resizing array and copying data)*

But resizing only occurs **rarely** (only when the number of elements is a power of 2). Hence, the expensive cost gets “spread out”:

- **Amortized Cost = O(1)**

Binary Representation of Integers

A binary number is a number expressed in a base 2. (unlike base 10, which we use in our daily life.)
Looking from right to left, the i th bit represents 2^i .

Base 10 example:

$1324 = 1 \cdot 10^3 + 3 \cdot 10^2 + 2 \cdot 10^1 + 4 \cdot 10^0 = 1000 + 100 + 20 + 4.$

Base 2 examples:

5 in base 2 is 101 in base 2: $1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 4 + 0 + 1.$

11 in base 2 is 1011 in base 2: $1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 0 + 2 + 1.$

Binary counter

A binary counter counts from 0 to $n - 1$ in base 2 representation.

- The least significant bit (LSB) flips every step.
- The second least bit flips every 2 steps.
- The third bit every 2^2 steps.

In general, the i th bit flips every 2^{i-1} steps.

The total number of operations is the total number of flips.

When counting from 0 to n (using $k = \lceil \log n \rceil$ bits)

Total number of flips $= n + \lfloor n / 2^1 \rfloor + \lfloor n / 2^2 \rfloor + \cdots + \lfloor n / 2^k \rfloor$

Example: counting from 0 to 7, using array size 3.

0 = [0,0,0]
1 = [0,0,1]
2 = [0,1,0]
3 = [0,1,1]
4 = [1,0,0]
5 = [1,0,1]
6 = [1,1,0]
7 = [1,1,1]

Binary counter – amortized analysis

What is the amortized cost for a single operation of a binary counter counting from 0 to n?

The total number of bits flips is: $n + \left\lfloor \frac{n}{2^1} \right\rfloor + \left\lfloor \frac{n}{2^2} \right\rfloor + \left\lfloor \frac{n}{2^3} \right\rfloor + \ldots + \left\lfloor \frac{n}{2^k} \right\rfloor$

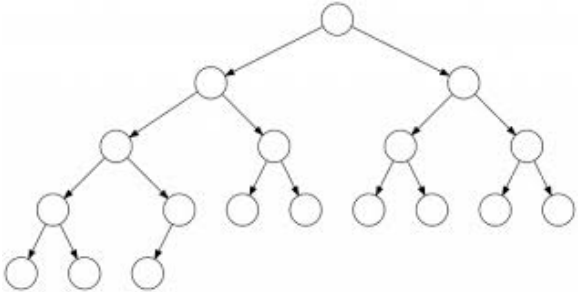
סדרה הנדסית

$$n + \left\lfloor \frac{n}{2^1} \right\rfloor + \left\lfloor \frac{n}{2^2} \right\rfloor + \left\lfloor \frac{n}{2^3} \right\rfloor + \ldots + \lfloor 1 \rfloor \leq n + \frac{n}{2^1} + \frac{n}{2^2} + \frac{n}{2^3} \ldots + 1 = n\left(1 + \frac{1}{2} + \frac{1}{4} + \ldots + \frac{1}{2^k}\right) \leq$$
$$n \cdot 2 = 2n$$

So, the amortized cost is $\frac{2n}{n} = 2 = O(1)$ = constant time.

Binary Heap

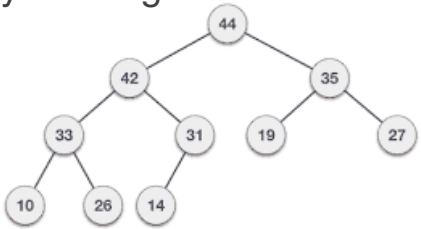
- A heap is a data structure that supports priority queue operations.
- A binary heap is an **almost complete** binary tree.
  - All leaves on the lowest level occur “to the left”.
  - All levels except the lowest one are completely filled.



Binary Heap

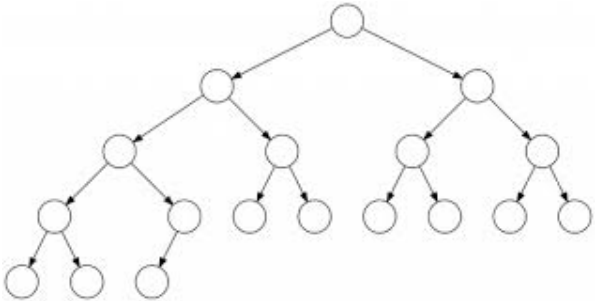
- In addition to being an almost complete binary, nodes in a binary heap satisfy the **heap property.**
- The element at each node is always smaller (or equal) to all its descendants.
- In this case, we call the structure a **min-heap.**
- Analogously, we could define a max-heap by requiring that the element at each node be bigger (or equal) than all of its descendants.
- We only draw the key associated with each element to simplify the diagrams. Elements can include any data so long as they have a key.

Example: max heap



Question 1

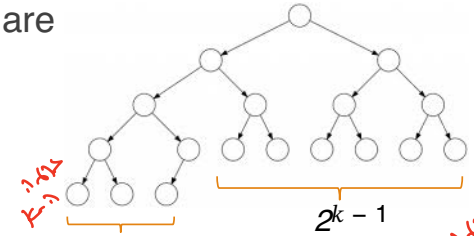
- **Prove:** in every almost complete binary tree with n vertices, exactly $\left\lceil \frac{n}{2} \right\rceil$ of the vertices are leaves.



Level	Depth
1	0
2	1
4	2
2^{k-1}	$k-1$

Question 1 – Solution

- The number of <u>leaves</u> in depth k is l (unknown)
- The number of <u>vertices</u> in depth $k-1$ is 2^{k-1}.
- Consider the nodes in depth $k-1$. The number of parents of the leaves in depth k is $\lceil \frac{l}{2} \rceil$, the remaining $2^{k-1} - \lceil \frac{l}{2} \rceil$ nodes are
- The total number of leaves is $l + (2^{k-1} - \left\lceil \frac{l}{2} \right\rceil)$
- The total number of <u>vertices</u> in the tree:
  - $n = 2^k - 1 + l = 2^k + l - 1$,
- If l is even, n is odd, $2^{k-1} + \frac{l}{2} = 2^{k-1} + \lceil \frac{l-1}{2} \rceil = \lceil \frac{n}{2} \rceil$
- If l is odd, n is even, $2^{k-1} + l - \lceil \frac{l}{2} \rceil = 2^{k-1} + \lfloor \frac{l}{2} \rfloor = \frac{n}{2}$



depth	#
0	1
1	2
2	4
k-1	2^{k-1}

Question 1 – Solution (Full expansion of even l)

- If l is even, then the total number of leaves is –

$$l + \left(2^{k-1} - \left\lceil \frac{l}{2} \right\rceil\right) = l + \left(2^{k-1} - \frac{l}{2}\right) = 2^{k-1} + \frac{l}{2} = 2^{k-1} + \left\lceil \frac{l-1}{2} \right\rceil$$

And by looking at the other side of the wanted equation –

$$\left\lceil \frac{n}{2} \right\rceil = \left\lceil \frac{2^k + l - 1}{2} \right\rceil = \left\lceil \frac{2^k}{2} \right\rceil + \left\lceil \frac{l-1}{2} \right\rceil = \left\lceil 2^{k-1} \right\rceil + \left\lceil \frac{l-1}{2} \right\rceil = 2^{k-1} + \left\lceil \frac{l-1}{2} \right\rceil$$

Value of n
From previous slide

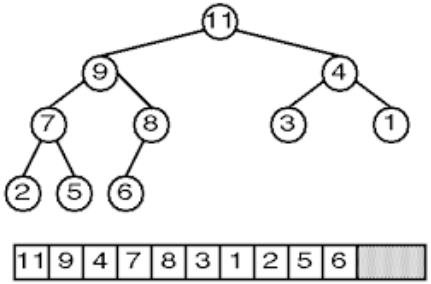
Because 2^k
Is even the ceil
can be split

We have reached the same equation on both sides, thus the total number of leaves is $\left\lceil \frac{n}{2} \right\rceil$ (in the case l is even, need to prove for odd l as well).

Heap – Array Implementation

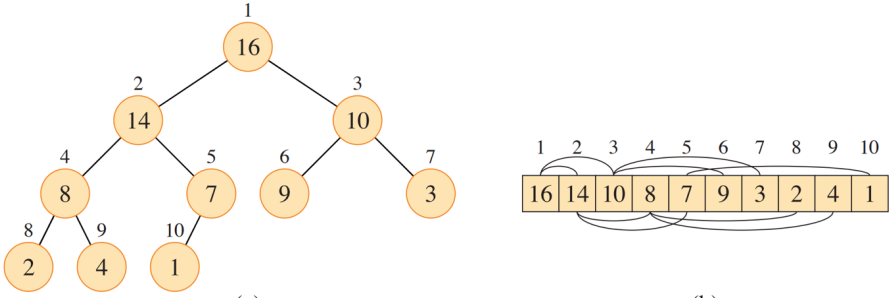
- The root is the first element of the array (index 1).
- For a node at location i, its left child will be stored at location $2i$, and its right child will be stored at location $2i + 1$.
- **Question:** Where can we find the parent of the node at the location i?

Note: we only show the keys (but the data is also stored)



Max heap implementation

- An **almost complete binary tree + an array**




```
PARENT(i)
return ⌊i/2⌋

LEFT(i)
return 2i

RIGHT(i)
return 2i + 1
```

Percolate up and down

- Two important operations on a heap are:
  - **Percolate up** – Move a node up the tree, as long as needed, until it reaches a valid location (where the node satisfies the max—heap property)
  - **Percolate down** – Move a node down the tree, as long as needed, until it reaches a valid location.

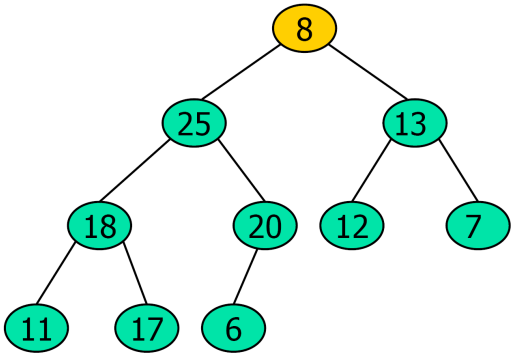
```
PercolateDown(i): [for a max-heap]
    while(A[i] is smaller than one of its children)
        j ← index of maximal child
        switch A[i] and A[j]
        i ← j
```

```
PercolateUp(i): [for a max-heap]
    while(A[i] is bigger than its parent)
        j ← index of parent
        switch A[i] and A[j]
        i ← j
```

Both methods visit at most one node in each level of the tree.

Percolate down

How will the tree look after percolating down the root?



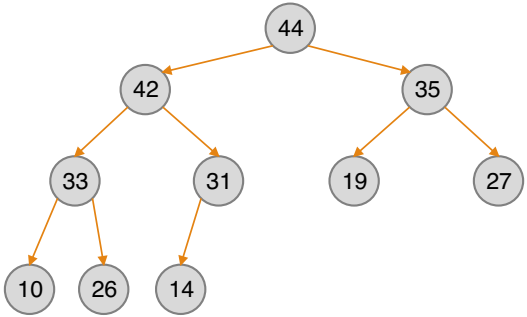

```
PercolateDown(i):
    while(A[i] is smaller than one of its children)
        j ← index of maximal child
        switch A[i] and A[j]
        i ← j
```

Common operations (max heap)

- **find-max():**
  - Return the value of the root.
  - Runs in time O(1).
- **delete-max():**
  - Delete the element with the maximum key and return it.
    - Put the last element in place of the root.
    - Percolate down the root.
- **insert(x, k):**
  - Insert element x with key k as the last element.
    - Percolate up the last element.

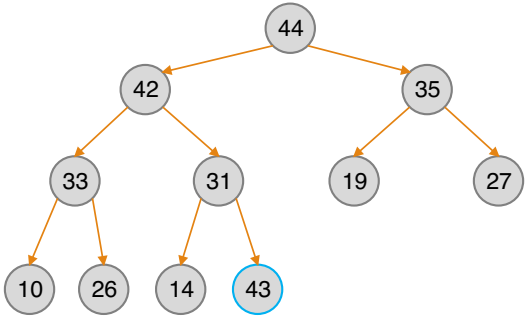
Add an Element - Example

- What will the heap look like after adding 43?



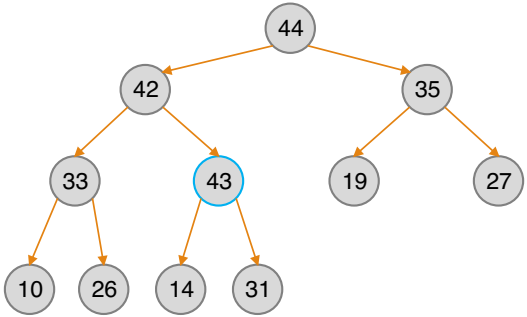
Add an Element - Example

- What will the heap look like after adding 43?



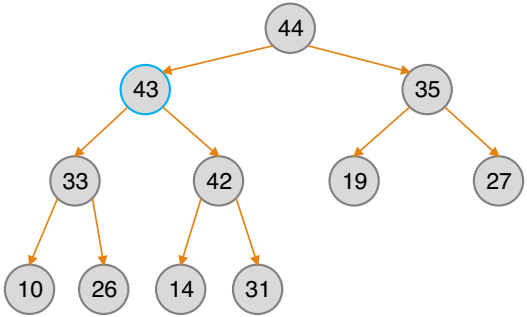
Add an Element - Example

- What will the heap look like after adding 43?



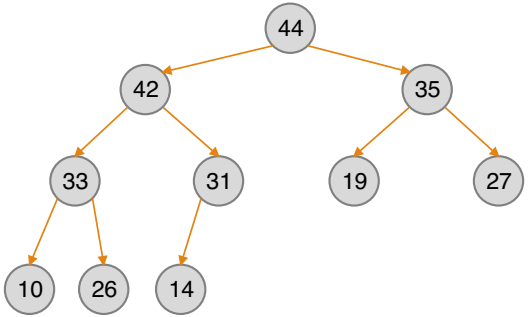
Add an Element - Example

- What will the heap look like after adding 43?



Extract Maximum - Example

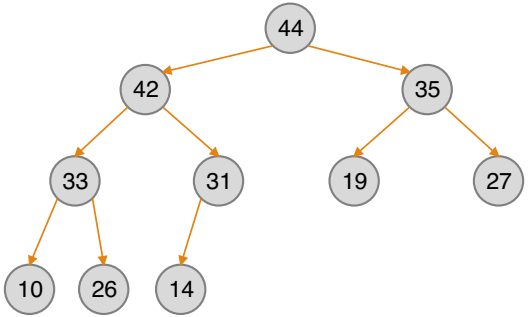
- What will the heap look like after extracting the maximum, twice?



Extract Maximum - Example

- What will the heap look like after extracting the maximum, twice?

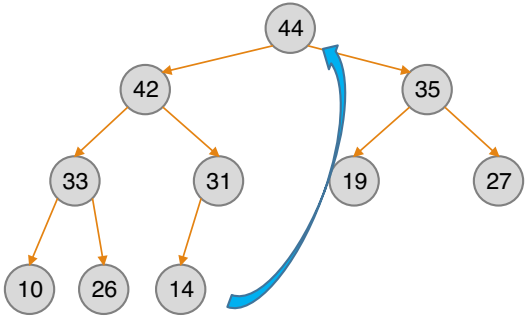
Copy max element: 44



Extract Maximum - Example

- What will the heap look like after extracting the maximum, twice?

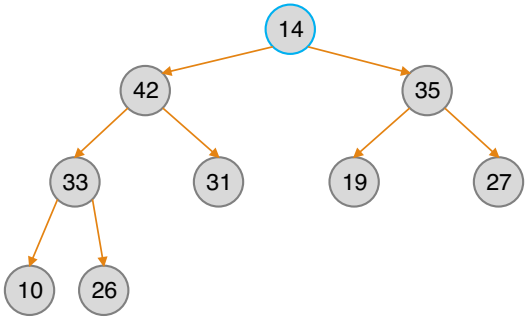
Copy max element: 44



Extract Maximum - Example

- What will the heap look like after extracting the maximum, twice?

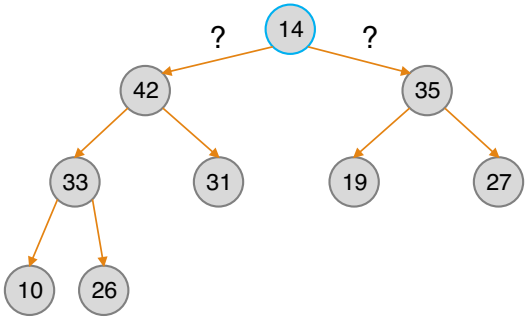
Copy max element: 44



Extract Maximum - Example

- What will the heap look like after extracting the maximum, twice?

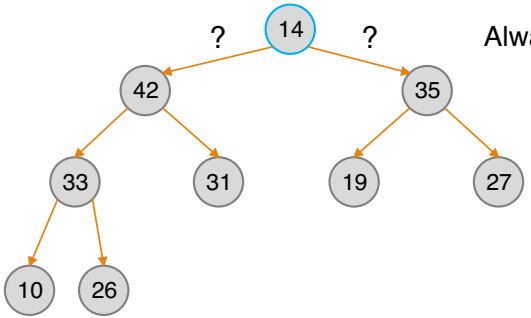
Copy max element: 44



Extract Maximum - Example

- What will the heap look like after extracting the maximum, twice?

Copy max element: 44

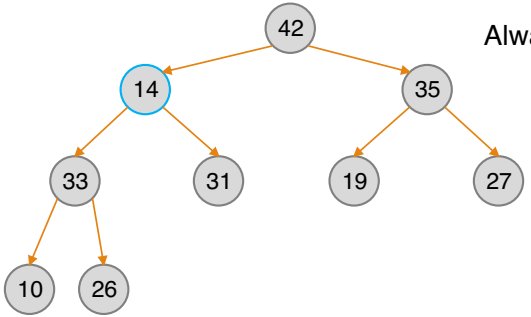


Always choose the **bigger** child

Extract Maximum - Example

- What will the heap look like after extracting the maximum, twice?

Copy max element: 44

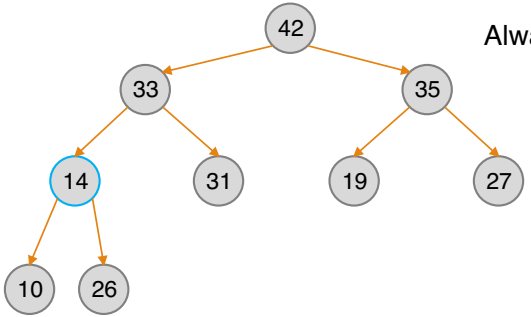


Always choose the **bigger** child

Extract Maximum - Example

- What will the heap look like after extracting the maximum, twice?

Copy max element: 44

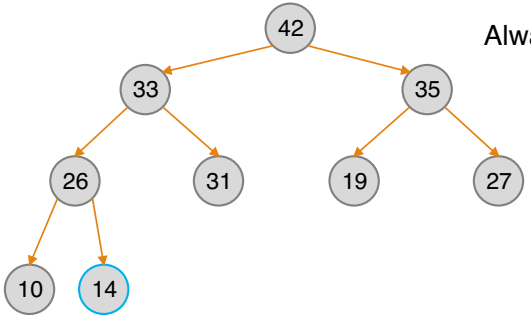


Always choose the **bigger** child

Extract Maximum - Example

- What will the heap look like after extracting the maximum, twice?

Copy max element: 44

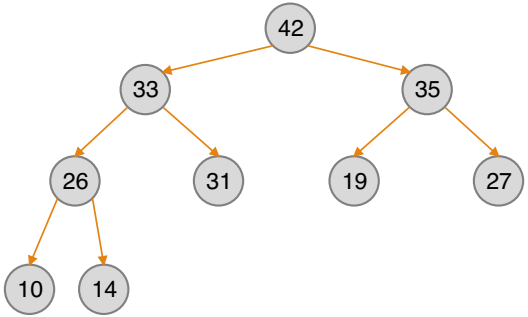


Always choose the **bigger** child

Extract Maximum - Example

- What will the heap look like after extracting the maximum, twice?

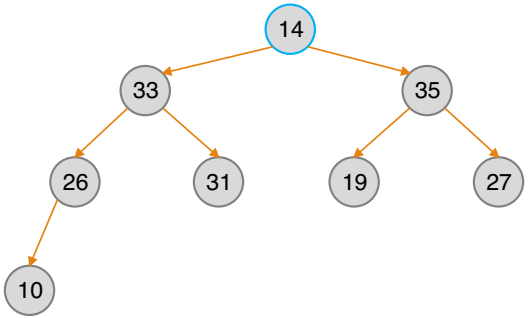
Copy max element: 44



Extract Maximum - Example

- What will the heap look like after extracting the maximum, twice?

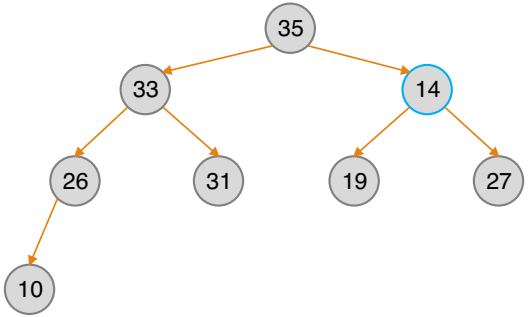
Copy max element: 44 42



Extract Maximum - Example

- What will the heap look like after extracting the maximum, twice?

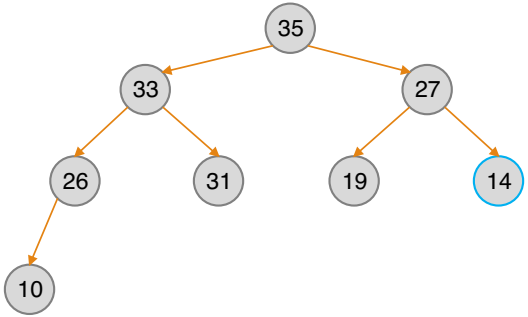
Copy max element: 44 42



Extract Maximum - Example

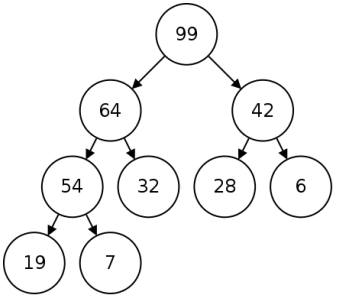
- What will the heap look like after extracting the maximum, twice?

Copy max element: 44 42



Question 2.1

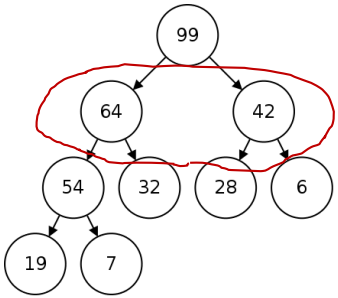
- Finding the value of the maximal element in the heap is trivial.
- How can we find the value of the second largest element?



Question 2.1

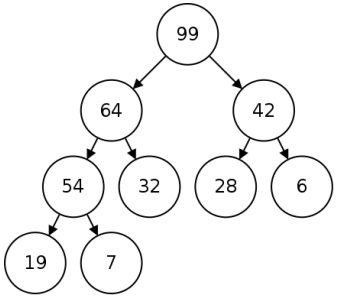
- Finding the value of the maximal element in the heap is trivial.
- How can we find the value of the second largest element?

- **Answer:**
- It must be a child of the root, one of the nodes in the second level.
- Just compare the two candidates.
- Time complexity: $O(1)$.



Question 2.2

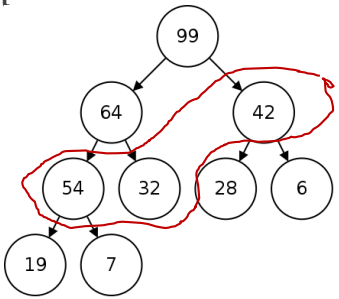
- How about the third largest?



Question 2.2

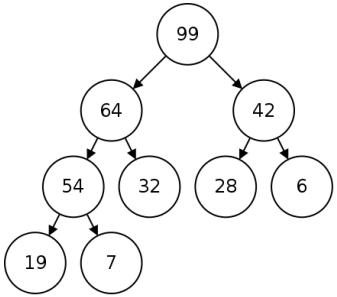
- How about the third largest?
- This time it could either be in the second level, but also on the third level.

- **Answer:**
- The third largest element is either a child of the largest element (not counting the second largest element) or a child of the second largest element.
- Just compare the three candidates.
- Time complexity: O(1).



Question 2.3

- Given $1 \leq k \leq n$ find the value of the k-th largest element.



Question 2.3

- Given $1 \leq k \leq n$, find the value of the k-th largest element.

- **Answer 1:**
- Extract the maximal element k times.
- This process destroys the heap.
- Thus, the k largest elements must be stored and inserted back into the heap after finding the required value.
- What is the runtime?
  - $O(k \log n)$ (k extractions + k insertions, each takes $O(\log n)$)

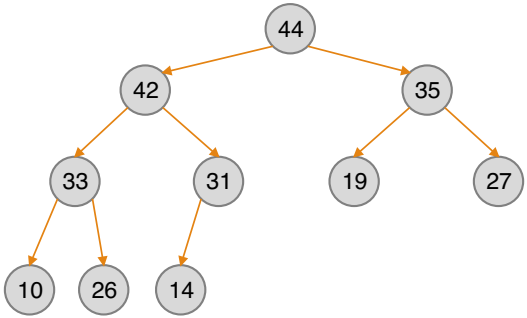
Question 2.3

- Given $1 \leq k \leq n$, find the value of the k-th largest element.

- **Answer 2:**
- As before, we observe that the k-largest element must be a child of one of the $k - 1$ largest ones.
- Suppose we know the $k - 1$ largest elements. All we need to do now is compare their children (only those who are not among the largest $k - 1$).
- Choose the largest one.
- We will now describe an algorithm for finding the k-th largest element by first finding **all** $k - 1$ largest elements.

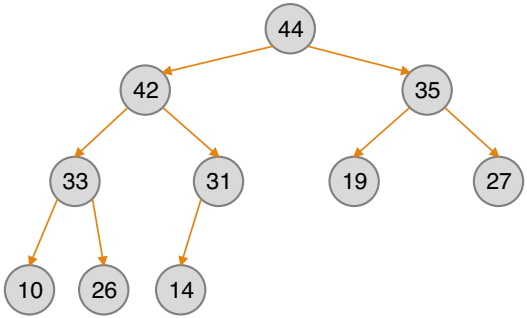
Solution 2

- Child collection



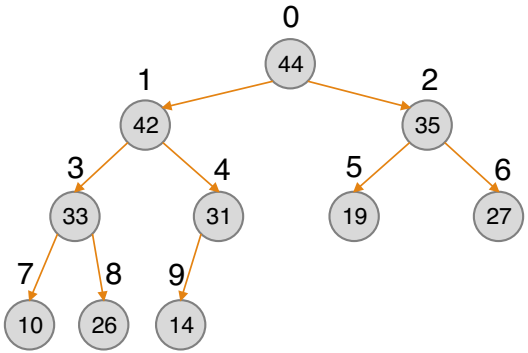
Solution 2

- **Child collection**
- Create a list of pointers to the heap



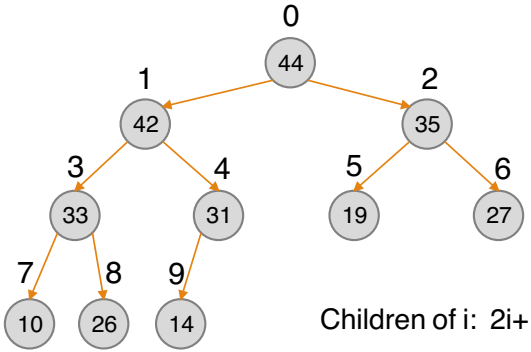
Solution 2

- **Child collection**
- Create a list of pointers to the heap



Solution 2

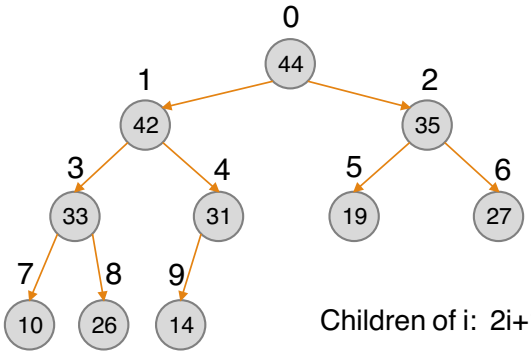
- **Child collection**
- Create a list of pointers to the heap



Children of i: 2i+1, 2i+2

Solution 2

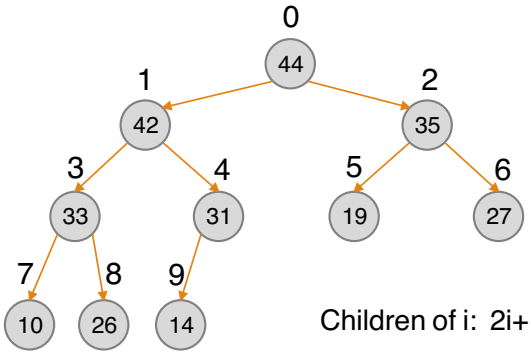
- **Child collection**
- Create a list of pointers to the heap
- **Step 1:** initiate the collection with a pointer to the root



Children of i: 2i+1, 2i+2

Solution 2

- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- **Step 2:** repeat k-1 times:

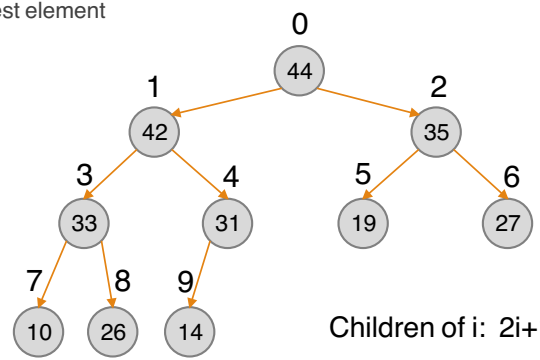


0

Children of i: 2i+1, 2i+2

Solution 2

- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- **Step 2:** repeat k-1 times:
  - Find the pointer to the biggest element

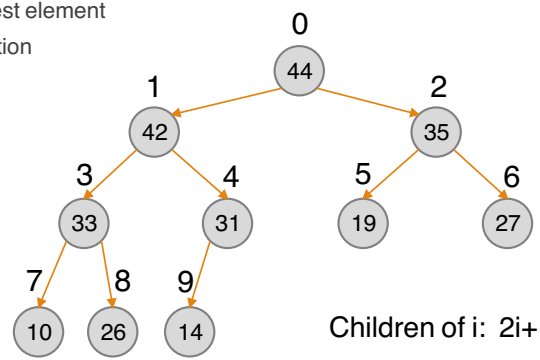


0

Children of i: 2i+1, 2i+2

Solution 2

- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- **Step 2:** repeat k-1 times:
  - Find the pointer to the biggest element
  - Remove pointer from collection

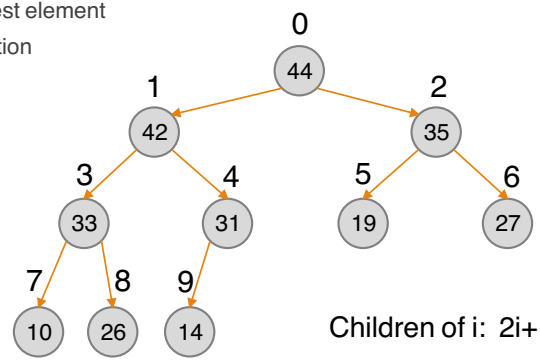


0

Children of i: 2i+1, 2i+2

Solution 2

- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- **Step 2:** repeat k-1 times:
  - Find the pointer to the biggest element
  - Remove pointer from collection
  - Add its children

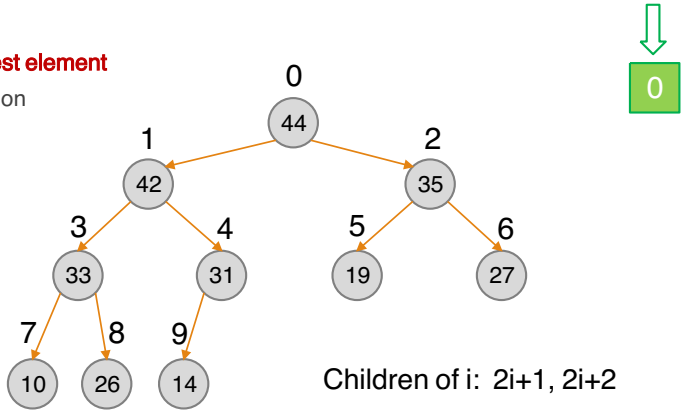


0

Children of i: 2i+1, 2i+2

Solution 2

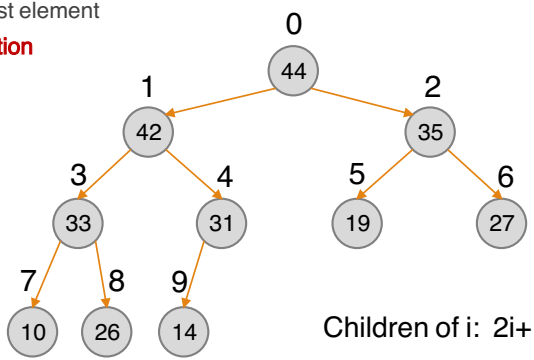
- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- **Step 2:** repeat k-1 times:
  - **Find the pointer to the biggest element**
  - Remove pointer from collection
  - Add its children



Children of i: 2i+1, 2i+2

Solution 2

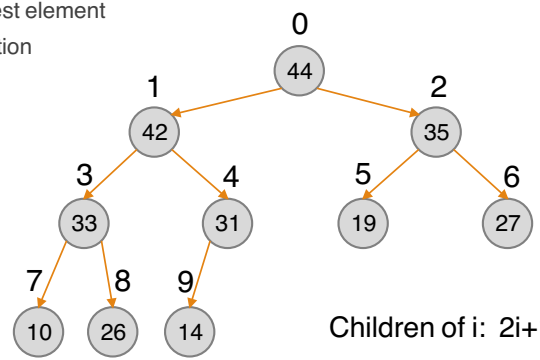
- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- **Step 2:** repeat k-1 times:
  - Find the pointer to the biggest element
  - **Remove pointer from collection**
  - Add its children



Children of i: 2i+1, 2i+2

Solution 2

- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- **Step 2:** repeat k-1 times:
  - Find the pointer to the biggest element
  - Remove pointer from collection
  - **Add its children**

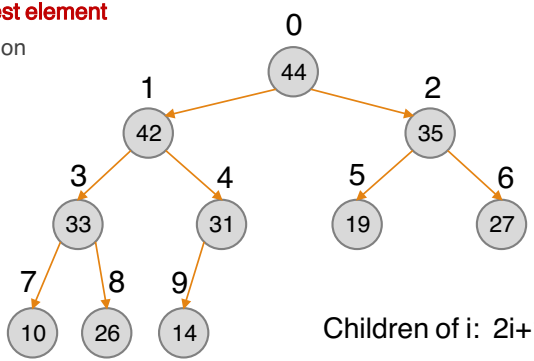


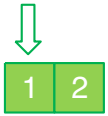
1	2

Children of i: 2i+1, 2i+2

Solution 2

- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- **Step 2:** repeat k-1 times:
  - **Find the pointer to the biggest element**
  - Remove pointer from collection
  - Add its children

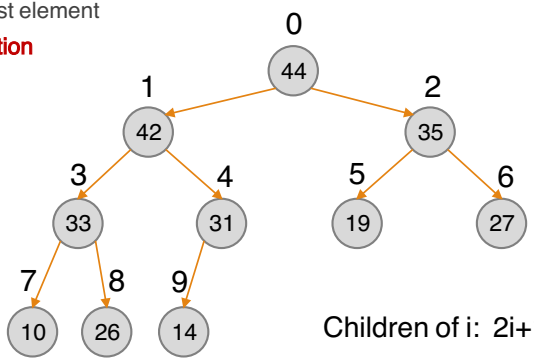




Children of i: 2i+1, 2i+2

Solution 2

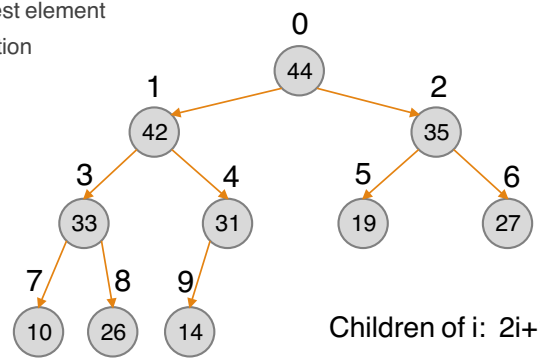
- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- **Step 2:** repeat k-1 times:
  - Find the pointer to the biggest element
  - **Remove pointer from collection**
  - Add its children



Children of i: 2i+1, 2i+2

Solution 2

- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- **Step 2:** repeat k-1 times:
  - Find the pointer to the biggest element
  - Remove pointer from collection
  - **Add its children**

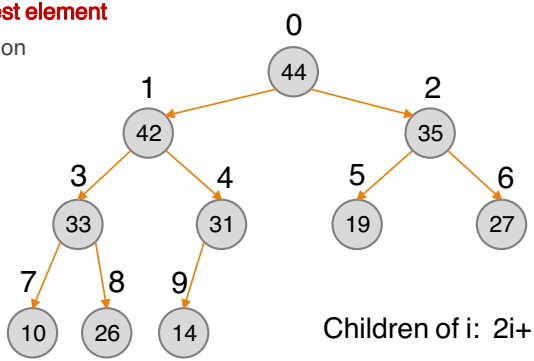


3	2	4

Children of i: 2i+1, 2i+2

Solution 2

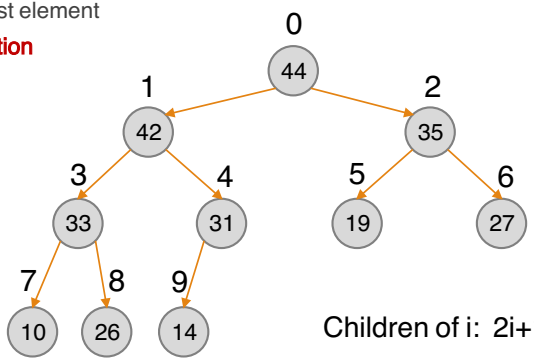
- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- **Step 2:** repeat k-1 times:
  - **Find the pointer to the biggest element**
  - Remove pointer from collection
  - Add its children



Children of i: 2i+1, 2i+2

Solution 2

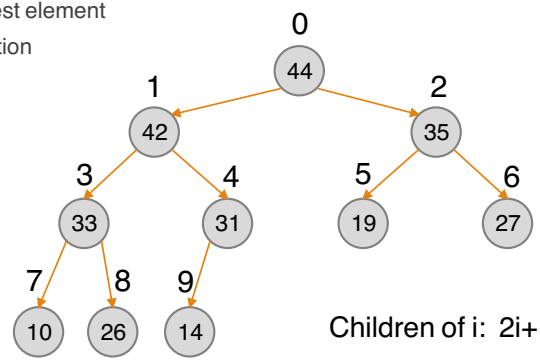
- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- **Step 2:** repeat k-1 times:
  - Find the pointer to the biggest element
  - **Remove pointer from collection**
  - Add its children



Children of i: 2i+1, 2i+2

Solution 2

- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- **Step 2:** repeat k-1 times:
  - Find the pointer to the biggest element
  - Remove pointer from collection
  - **Add its children**

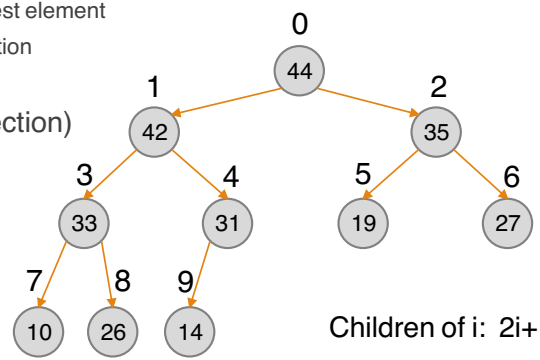


3	5	4	6

Children of i: 2i+1, 2i+2

Solution 2

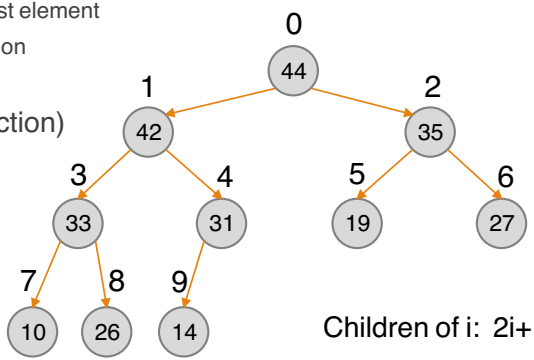
- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- Step 2: repeat k-1 times:
  - Find the pointer to the biggest element
  - Remove pointer from collection
  - Add its children
- **Step 3:** return max(collection)



Children of i: 2i+1, 2i+2

Solution 2

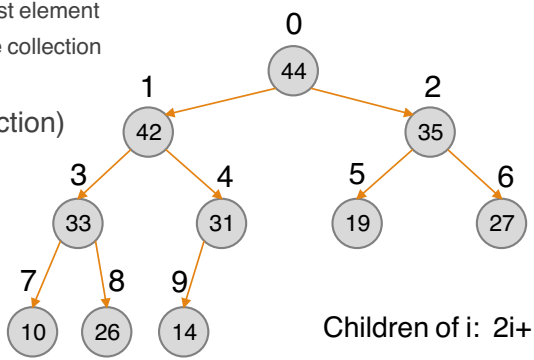
- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- Step 2: repeat k-1 times:
  - Find the pointer to the biggest element
  - Remove pointer from collection
  - Add its children
- Step 3: return max(collection)
- Complexity?



Children of i: 2i+1, 2i+2

Solution 2

- **Child collection**
- Create a list of pointers to the heap
- Step 1: initiate the collection with a pointer to the root
- Step 2: repeat k-1 times:
  - Find the pointer to the biggest element
  - Remove the pointer from the collection
  - Add it's children
- Step 3: return max(collection)
- Complexity?
  - **For the *i* 's iteration:**
  - Find max: O(i)
  - Remove + Add: O(1)
  - **Total O(k^2)**



Children of i: 2i+1, 2i+2

Question 2.3 – Improvement of the solution

- Can we improve on the construction above?

Question 2.3 - Improvement of the solution

- Can we improve on the construction above?
- Yes – We will keep the **candidates** in an additional binary heap.
- Init: Initialize an empty heap, B, contains only the root.
- At each iteration i : (perform the operations on heap B)
  - M is a max
  - *FindMax* (add to output), *DeleteMax*, *Insert* to B the two children of M
  - The number of elements (candidate to be max) in B before iteration i is i .
  - Cost of iteration i is O(log(i)) – 3 heap operations.

Total cost: O(k) + $\sum_{i=2}^{k}$O(log (i)) ≤ O(k) + $\sum_{i=2}^{k}$O(log (k)) = $O(k \cdot \log(k))$

Queue (ADT) - Reminder

- Linear order: insert at one end of *List*, remove at the other end.
- Queues are “FIFO” – first in, first out.
- A queue ensures “fairness”.
- Two main operations:
  - Enqueue, which adds an element to the one end of *List*, and
  - Dequeue, which removes the element at the other end.

Stack (ADT) - Reminder

- Linear order: the push and pop operations occur only at one end of *List*.
- Stacks are “LIFO” – last in, first out.
- Two main operations:
  - Push, which adds an element to the *List*, and
  - Pop, which removes the most recently added element that was not yet removed.

Question 3- If time permits

- Suggest a way to implement a queue using two stacks.
- You may only use the basic stack operations.
- Try to keep runtime to a minimum.

Question 3 - Solution

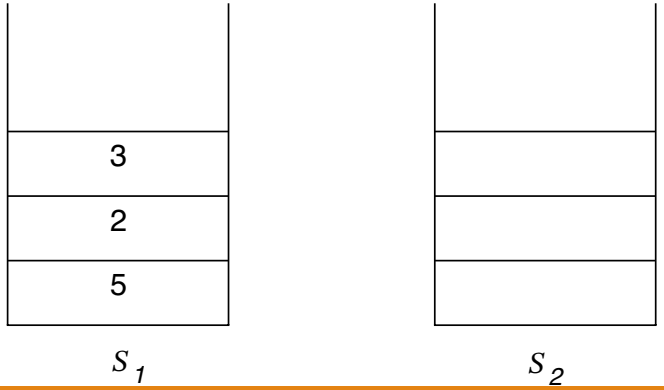
- We will use two stacks S_1 and S_2.
- Items are always pushed to S_1 (using push operation).
- Items are removed from S_2 (pop operation).
- If S_2 is empty – move all the elements in S_1 to S_2 by pop and push, and then pop an element from S_2.

Notice that popping and pushing the elements reverses their order.

Example

- Enqueue(5)
- Enqueue(2)
- Enqueue(3)

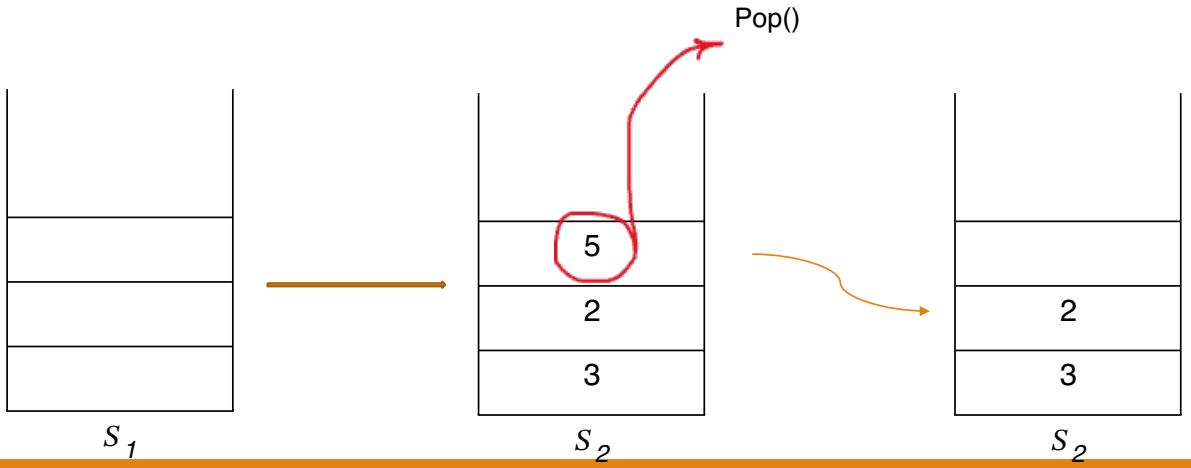
After performing the above three enqueue operations, the queue would look like this:



Example

- Dequeue()

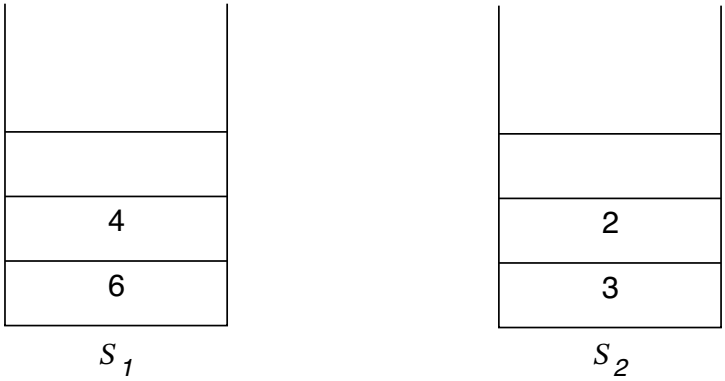
After performing the above dequeue operation, the queue would look like this:



Example

- Enqueue(6)
- Enqueue(4)

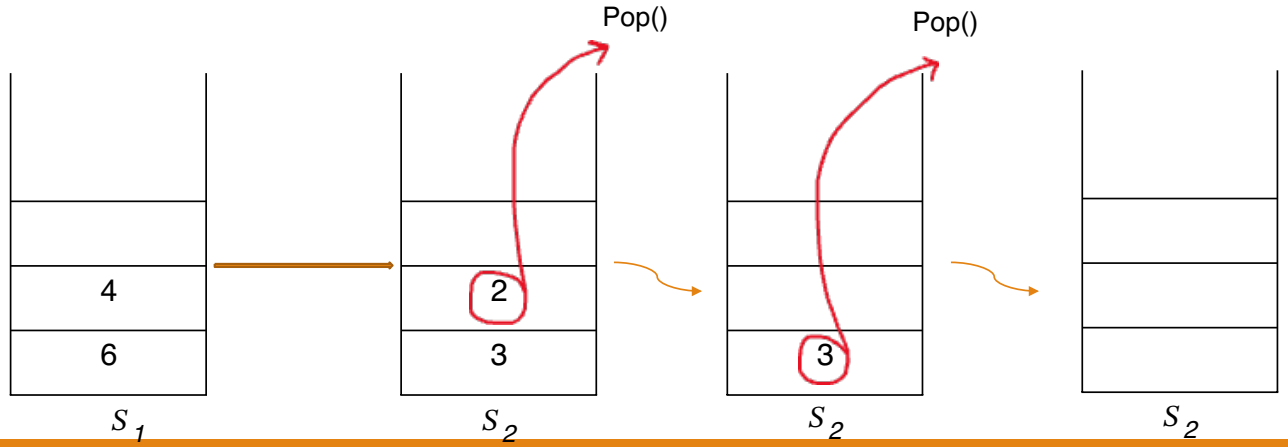
After performing the above enqueue operations, the queue would look like this:



Example

- Dequeue()
- Dequeue()

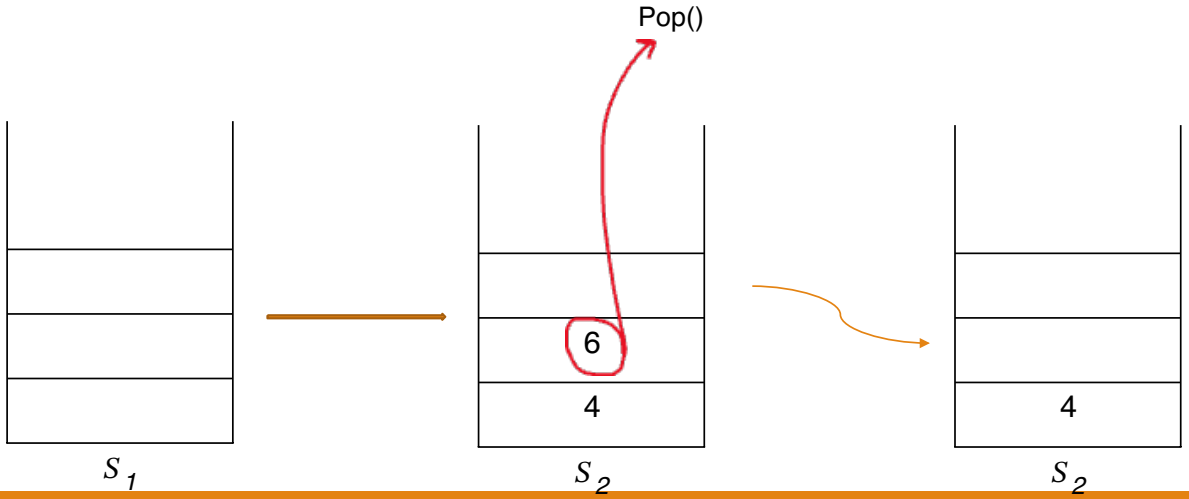
After performing the above dequeue operations, the queue would look like this:



Example

- Dequeue()

After performing the above dequeue operation, the queue would look like this:



Question 3 - Solution

- **enqueue**(x):
  - S_1.push(x)

- **dequeue**():
  - if (S_2.isEmpty()):
    - while(!S_1.isEmpty()):
      - S_2.push(S_1.pop())
  - S_2.pop()

Time complexity:

<u>Enqueue</u>:

O(1)

<u>Dequeue</u>:

The while loop may have many iterations, however, every element is moved from S_1 to S_2 at most once during the whole operation of the data structure, therefore, the amortized cost of dequeue is O(1).